

LAB SESSION 7

Programming Exercise:

Question 1: Create a base class **Vehicle** with a method **move()** that prints "Vehicle is moving". Derive a class **Car** from **Vehicle** and override the **move()** method to print "Car is moving".

```
C:\C++ PROGRAMS> g++ tut1.cpp > ...
1  #include <iostream>
2  using namespace std;
3
4  class Vehicle
5  {
6  public:
7      // Make the method virtual to allow overriding
8      virtual void move()
9      {
10         cout << "Vehicle is moving" << endl;
11     }
12 };
13
14 class Car : public Vehicle
15 {
16 public:
17     // Override the move method
18     void move() override
19     {
20         cout << "Car is moving" << endl;
21     }
22 };
23
24 int main()
25 {
26     Vehicle v;
27     v.move(); // Vehicle's move
28
29     Car c;
30     c.move(); // Car's move
31
32     return 0;
33 }
```

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Vehicle is moving
Car is moving
PS C:\C++ PROGRAMS>
```

Question 2: Constructor in Derived Class Create a base class **Person** with attributes **name** and **age**. Create a derived class **Student** with an additional attribute **studentID**. Initialize these attributes using constructors.

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Person Constructor Called
Student Constructor Called
Name: John Doe, Age: 20
Student ID: 12345
PS C:\C++ PROGRAMS>
```

```

#include <iostream>
using namespace std;

// Base class Person
class Person {
protected:
    string name;
    int age;

public:
    // Constructor to initialize name and age
    Person(string n, int a) : name(n), age(a) {
        cout << "Person Constructor Called" << endl;
    }

    // Method to display person details
    void displayPerson() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

// Derived class Student
class Student : public Person {
private:
    int studentID;

public:
    // Constructor to initialize name, age, and studentID
    Student(string n, int a, int id) : Person(n, a), studentID(id) {
        cout << "Student Constructor Called" << endl;
    }

    // Method to display student details
    void displayStudent() {
        displayPerson(); // Call base class method to display person
        details cout << "Student ID: " << studentID << endl;
    }
};

int main() {
    // Create an instance of Student
    Student s("John Doe", 20, 12345);

    // Display the details of the student
    s.displayStudent();

    return 0;
}

```

Question 2: Constructor in Derived Class Create a base class Person with attributes name and age. Create a derived class Student with an additional attribute studentID. Initialize these attributes using constructors.

```
#include <iostream>
using namespace std;

// Base class
class Person {
protected:
    string name;
    int age;

public:
    // Constructor for Person
    Person(string n, int a) {
        name = n;
        age = a;
    }

    // Function to display Person details
    void displayPerson() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

// Derived class
class Student : public Person {
private:
    int studentID;

public:
    // Constructor for Student
    Student(string n, int a, int id) : Person(n, a)
    {
        studentID = id;
    }

    // Function to display Student details
    void displayStudent() {
        displayPerson(); // Call base class function
        cout << "Student ID: " << studentID << endl;
    }
};

int main() {
    // Create a Student object
    Student s("Ali", 20, 12345);

    // Display student details
    s.displayStudent();

    return 0;
}
```

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }
Name: Ali
Age: 20
Student ID: 12345
PS C:\C++ PROGRAMS>
```

Question 3: Method Overriding Create a base class Shape with a method draw() that prints "Drawing Shape". Create derived classes Circle and Square that override the draw() method to print "Drawing Circle" and "Drawing Square", respectively.

```
#include <iostream>
using namespace std;

// Base class
class Shape {
public:
    // Virtual function to allow overriding
    virtual void draw() {
        cout << "Drawing Shape" << endl;
    }
};

// Derived class Circle
class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle" << endl;
    }
};

// Derived class Square
class Square : public Shape {
public:
    void draw() override {
        cout << "Drawing Square" << endl;
    }
};

int main() {
    Shape* shape;          // Base class
    Circle c;
    Square s;

    // Pointing to Circle object
    shape = &c;
    shape->draw(); // Output: Drawing Circle
    // Pointing to Square object
    shape = &s;
    shape->draw(); // Output: Drawing Square
    return 0;
}
```

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }  
Drawing Circle  
Drawing Square  
PS C:\C++ PROGRAMS>
```

Question 4: Access Specifiers Create a base class Base with a private attribute privateVar, a protected attribute protectedVar, and a public attribute publicVar. Create a derived class Derived and demonstrate access to these attributes. Write your observations.

```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }  
Inside Base class:  
privateVar = 1  
protectedVar = 2  
publicVar = 3  
  
Inside Derived class:  
protectedVar = 2  
publicVar = 3  
  
Accessing publicVar directly from main: 3  
PS C:\C++ PROGRAMS>
```

```

#include <iostream>
using namespace std;

// Base class
class Base {
private:
    int privateVar;

protected:
    int protectedVar;

public:
    int publicVar;

    Base() {
        privateVar = 1;
        protectedVar = 2;
        publicVar = 3;
    }

    void showBase() {
        cout << "Inside Base class:" << endl;
        cout << "privateVar = " << privateVar << endl;
        cout << "protectedVar = " << protectedVar << endl;
        cout << "publicVar = " << publicVar << endl;
    }
};

// Derived class
class Derived : public Base {
public:
    void showDerived() {
        cout << "\nInside Derived class:" << endl;

        // cout << "privateVar = " << privateVar << endl; // ✗ Not accessible (will cause
        // error) // privateVar is not accessible in Derived class

        cout << "protectedVar = " << protectedVar << endl; // ✓ Accessible
        cout << "publicVar = " << publicVar << endl; // ✓ Accessible
    }
};

int main() {
    Derived d;
    d.showBase();
    d.showDerived();

    cout << "\nAccessing publicVar directly from main: " << d.publicVar << endl;

    // cout << d.protectedVar; // ✗ Not accessible here
    // cout << d.privateVar; // ✗ Not accessible here

    return 0;
}

```

Question 5 You are required to create a C++ program that demonstrates the concept of inheritance. Consider the following requirements:

Base Class - Shape:

Attributes:

- color (string)
- Constructor that initializes color.
- A pure virtual function area() which will return the area of the shape.
- A virtual function display() that prints the color of the shape.

Derived Classes:

Rectangle:

- Attributes: width (double), height (double)
- Constructor that initializes color, width, and height.
- Override area() to calculate and return the area of the rectangle.
- Override display() to print the color, width, height, and area of the rectangle.

Circle:

Attributes:

- radius (double)
- Constructor that initializes color and radius.
- Override area() to calculate and return the area of the circle.
- Override display() to print the color, radius, and area of the circle.

Main Function:

- Create instances of Rectangle and Circle with different attributes.
- Use a pointer to Shape to store the addresses of these instances and call their display() method.

```

#include <iostream>
#include <cmath>
using namespace std;

// Abstract Base Class
class Shape {
protected:
    string color;

public:
    Shape(string c) {
        color = c;
    }

    virtual double area() = 0; // Pure virtual function

    virtual void display() {
        cout << "Color: " << color << endl;
    }

    virtual ~Shape() {} // Virtual destructor for safety
};

// Derived class: Rectangle
class Rectangle : public Shape {
private:
    double width;
    double height;

public:
    Rectangle(string c, double w, double h) : Shape(c), width(w), height(h)
    {}

    double area() override {
        return width * height;
    }

    void display() override {
        cout << "Rectangle" << endl;
        cout << "Color: " << color << endl;
        cout << "Width: " << width << ", Height: " << height << endl;
        cout << "Area: " << area() << endl;
    }
};

// Derived class: Circle
class Circle : public Shape {
private:
    double radius;

public:
    Circle(string c, double r) : Shape(c), radius(r) {}

    double area() override {
        return 3.14159 * radius * radius;
    }

    void display() override {
        cout << "Circle" << endl;
        cout << "Color: " << color << endl;
        cout << "Radius: " << radius << endl;
        cout << "Area: " << area() << endl;
    }
};

// Main function
int main() {
    // Create Rectangle and Circle instances
    Rectangle rect("Blue", 4.5, 3.0);
    Circle circ("Red", 2.5);

    // Pointer to base class Shape
    Shape* shapePtr;

    cout << "--- Rectangle Info ---" << endl;
    shapePtr = &rect;
    shapePtr->display();

    cout << "\n--- Circle Info ---" << endl;
    shapePtr = &circ;
    shapePtr->display();

    return 0;
}

```



```
PS C:\Users\hp\Unity Projects\Flappy Bird> cd "c:\C++ PROGRAMS\" ; if ($?) { g++ tut1.cpp -o tut1 } ; if ($?) { .\tut1 }  
--- Rectangle Info ---  
Rectangle  
Color: Blue  
Width: 4.5, Height: 3  
Area: 13.5  
  
--- Circle Info ---  
Circle  
Color: Red  
Radius: 2.5  
Area: 19.6349  
PS C:\C++ PROGRAMS>
```